

What This Code Teaches — Design Principles in the Stack Overflow API

Every principle below is visible in the code you just wrote. Nothing here is theory — each one answers the same question: **why do senior engineers write code differently?**

1. Single Responsibility + Separation of Concerns

Every class has one reason to change, every layer one job:

```
QuestionController    → HTTP: parse, delegate, respond
QuestionService       → business rules
QuestionRepository    → data access
```

The test is a question, not a definition: *if tomorrow the storage changes from HashMap to Postgres, does `QuestionService` change?* No → concerns are separated. If email-sending were added to `QuestionService` tomorrow, would a change in email templates force a change there? Yes → SRP violated; extract it.

The controller never knows a `ConcurrentHashMap` exists. The repository never knows HTTP exists.

2. Dependency Inversion + Inversion of Control

Two related ideas, one line of code each.

DIP — depend on the abstraction:

```
java
private final QuestionRepository questions;    // ✓ interface
// NOT: private final InMemoryQuestionRepository questions; × concrete
```

Tomorrow: `HashMap → MySQL → MongoDB`. The service doesn't change a character.

IoC — dependencies are supplied from outside, not created inside:

```
java
// × the service constructing its own dependency
public QuestionService() { this.questions = new InMemoryQuestionRepository(); }

// ✓ ours: injected through the constructor
public QuestionService(QuestionRepository questions, AnswerRepository answers)
```

All the `new` lives in one place — `App.main`, the composition root. The dependency graph is assembled from the outside in. (This is exactly what Spring automates; you've now built it

by hand once, so the framework will never be magic to you.)

3. Encapsulation, Tell-Don't-Ask, and the Rich Domain Model

Three names for one instinct: **behavior belongs with the data it operates on.**

```
java

// × Ask: pull the data out, decide elsewhere – logic leaks into every caller
if (question.upvotes() - question.downvotes() < 0) { ... }

// ✓ Tell: the object owns its own rules
question.score()           // derived inside the record – callers can't get the
```

`score()` in our `Question` is a small rich-domain move: the up-minus-down rule exists in exactly one place. When you add voting (homework), continue the pattern:

```
java

question.upvote()           // ✓ the object protects its own invariants
question.score++            // × anyone can corrupt state from anywhere
```

A model with only getters — an **anemic** model — pushes every decision into services, which then bloat into the god-classes SRP warned about. Ask each new rule: *which object owns this data? Then that object owns this behavior.*

4. Law of Demeter — talk only to your immediate friends

```
java

// × Question now depends on Author AND Company AND Address AND City...
question.author().company().address().city().name();

// ✓ one dot: ask your friend, let the friend handle the rest
question.authorCity();
```

Every dot in a chain is a coupling: if `Company` restructures, `Question`'s caller breaks — a class it never should have known about. Real-world version: you don't ask a friend "go ask your father for his manager's wife's phone number" — you ask the friend, and the friend deals with it.

The smell is visual: `a.b().c().d()` — **many dots, many strangers**. In our code, notice the controller calls `questionService.list(filter)` and stops — it never reaches through the service into the repository (`questionService.getRepo().getMap()...` would be the violation).

(Caveat: chained calls on the SAME object — streams, builders — are fine. Demeter is about reaching through different objects, not about counting dots mechanically.)

5. Information Hiding

`InMemoryQuestionRepository` uses a `ConcurrentHashMap` and an `AtomicLong`. **Nobody else knows.** The interface exposes intent (`findAll(filter)`), never mechanism. Replace the map with Postgres and no caller notices — that's the definition working.

Corollary in the same file: `comparatorFor`, `sortValueOf`, `isAfter` are `private`. The sorting mechanics are the repository's secret.

6. Immutability + Defensive Copying

Every record in the system is immutable — and one line does more work than it appears to:

```
java

public Question {
    tags = List.copyOf(tags);    // defensive copy
}
```

Without it, this compiles and corrupts your "immutable" question from the outside:

```
java

List<String> tags = new ArrayList<>(List.of("java"));
Question q = new Question(..., tags, ...);
tags.add("hacked");           // q.tags() now contains "hacked" — no copyOf
```

A record guards its *references*, not the objects behind them. `List.copyOf` closes that hole (and returns an unmodifiable list, so `q.tags().add(...)` throws too).

Payoffs: thread-safe with zero locks, safe to cache, safe to share, and an "edit" becomes an explicit new value (`withId`) instead of invisible mutation.

7. Value Objects vs Entities

```
java

Question    → ENTITY: has identity. Two questions with the same title are DIFFERENT
              Equality means same id.

QuestionFilter, Cursor.Position, CursorPage
              → VALUE OBJECTS: no identity. Two filters with identical fields ARE EQUAL
              Equality means same values.
```

Java records give value objects their correct `equals`/`hashCode` for free — one reason records fit DTOs, filters, and cursors so naturally. Entities need id-based identity and a lifecycle; values are interchangeable and disposable.

8. The DTO Boundary + Transformer

Entities never cross the HTTP boundary; the seam between the two worlds is one static factory:

```
java
Question → QuestionSummaryResponse.from(q) → JSON
```

Keeping mapping in `from(...)` (a transformer) rather than inline in the service keeps the service reading like business logic:

```
java
return questions.findAll(filter).map(QuestionSummaryResponse::from);
```

One line: *repository decides how, transformer decides shape, service just composes.*

9. API Evolution + Postel's Law

Be conservative in what you send; be liberal in what you accept.

Everything in this codebase that looks like "extra ceremony" exists so the API can change without breaking clients:

Mechanism in our code	What it lets us change later
DTOs instead of entities	rename/refactor <code>Question</code> internals freely
<code>from(...)</code> transformers	reshape responses in one place
Opaque Base64 cursor	switch cursor internals (add fields, change encoding) — clients just echo the token
Ignoring unknown JSON fields (Jackson default)	clients can send tomorrow's fields today
<code>score</code> computed, not stored	change the ranking formula without a migration

Adding `bookmarkCount` to `QuestionSummaryResponse` tomorrow breaks nobody — old clients ignore the new field, and no internal type leaked that could have pinned us down.

10. HTTP Semantics — Safe, Idempotent, PUT vs PATCH

Method choice is a design decision, not a formality:

SAFE	(no state change):	GET, HEAD, OPTIONS
IDEMPOTENT	(n calls = 1 call):	GET, PUT, DELETE
NEITHER:		POST

- `POST /questions` — creates; calling twice creates two questions. Correct: creation is not idempotent.
- `PUT /questions/1/vote` (homework) — *replaces* your vote; calling twice still yields +1, not +2. PUT is the right verb precisely **because** it promises idempotency; a retried timeout must not double-count.
- `PUT` replaces the whole resource; `PATCH /questions/10` with `{"title": "Updated"}` modifies one field without resending body and tags.

Idempotency isn't HTTP trivia — it's how you survive retries in a distributed world.

11. Cursor Pagination — because it scales, not because it's cool

```
offset → DB reads-and-discards 10,000 rows; pages drift under inserts
cursor → "the last row I saw was (value, id); give me the next 20" - index
seek, drift-free
opaque → Base64 so clients can't depend on (and freeze) the internals
```

And the detail that separates working code from correct code — the tie-breaker:

```
java
base.thenComparing(Question::id, Comparator.reverseOrder());
```

Sort keys (timestamps, scores) aren't unique; without a unique tie-breaker in both the *ordering* and the *cursor*, pages silently skip or repeat rows that share a value. The cursor is a `(sortValue, id)` pair for the same reason.

The five that change how you think

1. **Law of Demeter** — stop navigating object graphs; ask your friend and let the friend handle it.
2. **Tell, Don't Ask** — move decisions into the object that owns the data.
3. **DIP + IoC** — depend on abstractions; let the outside supply them.
4. **Rich Domain Model** — behavior lives with data, not in getter-farms plus god-services.
5. **Postel's Law** — strict in what you produce, tolerant in what you accept; that's what lets an API live for years.

The syntax of Javalin, records, and streams will fade. These five won't.